DS HOMEWORK EX.no.4

1.implement a stack using queue data structure

```
#include <stdio.h>
#include<stdlib.h>
#define emptyq -1

struct queue
{
   int front;
   int rear;
   int *arr;
   int cap;
};

 typedef struct queue *queue;

 int isempty(queue q)
 {
    return(q->front==emptyq);
 }
 int isfull(queue q)
 {
```

```
    return(q->rear==q->cap-1);
 }

void makeempty(queue q)
{
   q->front=emptyq;
   q->rear=emptyq;
}

queue create(int max)
{
   queue q;
   q=(queue)malloc(sizeof(struct queue));
   if(q!=NULL)
   {
      q->arr=(int*)malloc(max*sizeof(int));
      q->cap=max;
      makeempty(q);
   }
   return q;
}

queue enqueue(queue q,int x)
```

```
{
   if(isfull(q))
   {
      printf("\nqueue is full");
   }
   else if(isempty(q))
   {
      q->front++,q->rear++;
      q->arr[q->front]=q->arr[q->rear]=x;
   }
   else
   {
      q->rear++;
      q->arr[q->rear]=x;
   }
   return q;
}
int dequeue(queue q)
{
   int val;
   if(isempty(q))
   {
      printf("\nqueue is empty");
```

```
        return -1;
    }
    else if(q->front==q->rear)
    {
        val=q->arr[q->front];
        makeempty(q);
    }
    else
    {
         val=q->arr[q->front];
        q->front++;
    }

    return val;
    }




int topvalue(queue q)
{
    return(q->arr[q->front]);
}
```

```
queue push(queue q,int x)
{
   enqueue(q,x);
   int size=q->rear-q->front+1;
   for(int i=0;i<size;i++)
   {
      enqueue(q,dequeue(q));
   }
   return q;
}

int pop(queue q)
{
   int val=q->arr[q->front];
   dequeue(q);
   return val;
}

void display(queue q)
{
   int i;
   for(i=q->front;i<=q->rear;i++)
```

```
    {
        printf("\t%d",q->arr[i]);
    }
}
int main()
{

    queue q;
    q=create(20);
    push(q,20);
    push(q,80);
    push(q,60);
    display(q);
    printf("\ndeleted element=%d",pop(q));
    printf("\ntop value=%d",topvalue(q));

    return 0;
}
```

```
        60      80      20
deleted element=60
top value=80

...Program finished with exit code 0
Press ENTER to exit console.
```

2.implement queue using stack data structure

```
#include<stdio.h>
#define max 100

int top=-1,stack[max];

void push(int x)
{
   top++;
   stack[top]=x;
}
int pop()
{
   int val=stack[top];
   top--;
   return val;
}

void enqueue(int x)
{
   int i=-1;
   int temp[max];
   if(top==max-1)
   {
```

```
        printf("\nqueue is full");
    }


    while(top!=-1)
    {
        i++;
         temp[i]=pop();
        ;


    }
    push(x);


    while(i!=-1)
    {
        push(temp[i]);
        i--;
    }
    }


int dequeue()
{
```

```
    if(top==-1)
    {
        printf("\nqueue is empty");
        return -1;
    }
    else
    {
        int val=stack[top];
        top--;
        return val;
    }
}

void display()
{
    int i;
    for(i=top;i>=0;i--)
    {
        printf("%d\t",stack[i]);
    }
}

int main()
```

```
{
   int x;
   int n;
   printf("\nEnter n=");
   scanf("%d",&n);
   for(int i=0;i<n;i++)
   {
      scanf("%d",&x);
      enqueue(x);
   }
   display();
  printf("\ndeleted elt=%d",dequeue());
   printf("\ndeleted elt=%d\n",dequeue());
   display();
   return 0;

}
```

```
Enter n=3
1
2
3
1       2       3
deleted elt=1
deleted elt=2
3
```

3.write a c program using stack data structure to implement managing the history of web page

```
#include<stdio.h>

#include<string.h>

#define max 100


int top=-1;

char stack[max][50];


void push(char url[])

{

   if(top==max-1)

   {

      printf("\nhistory is full");

   }

   else

   {

   top++;

 strcpy(stack[top],url);

   }

}

char pop()

{

   if(top==-1)
```

```
    {
        printf("\nno previous page");
    }
    printf("\nleaving site:%s",stack[top]);
    top--;
}



void display()
{
    int i;
    printf("\nBrowsing history:");
    for(i=top;i>=0;i--)
    {
        printf("\n%s",stack[i]);
    }
    printf("\n-------------------\n");
}
void peek()
{
    printf("\ncurrent page=%s",stack[top]);
}
```

```
int main()
{

  push("youtube.com");
  push("instagram.com");
  push("spotify.com");
  push("facebook.com");
  display();
  peek();
  pop();
   display();
   peek();
   return 0;

}
```

```
Browsing history:
facebook.com
spotify.com
instagram.com
youtube.com
-------------------

current page=facebook.com
leaving site:facebook.com
Browsing history:
spotify.com
instagram.com
youtube.com
-------------------

current page=spotify.com
```

4.write a c program to sort a stack using temproray stack

```
#include<stdio.h>

#define max 100


int top1=-1,top2=-1, stack1[max],stack2[max];

void push1(int x)

{

   if(top1==max-1)

   {

      printf("\nstack is full");

   }

   else

   {
```

```
        top1++;
        stack1[top1]=x;

    }
}
void push2(int x)
{
    if(top2==max-1)
    {
        printf("\nstack is full");
    }
    else
    {
        top2++;
        stack2[top2]=x;

    }
}
int pop1()
{
    if(top1==-1)
    {
        printf("\nstack is empty");
```

```
        return -1;
    }
    else
    {
        int val=stack1[top1];
        top1--;
        return val;
    }
}
int pop2()
{
    if(top2==-1)
    {
        printf("\nstack is empty");
        return -1;
    }
    else
    {
        int val=stack2[top2];
        top2--;
        return val;
    }
}
```

```
void sort()
{
   int x,small,index=0;
   int temp[max];
   int n=top1;
   while(n>0)
   {
   small=stack1[top1];
   while(top1!=-1)
   {
      x=pop1();
      if(small>x)
      {
         small=x;
      }


   push2(x);
   }
   int found=0;
   while(top2!=-1)
   {
```

```
        int x=pop2();
        if(small==x&&!found)
        {
           found=1;
        }
        else
        {
        push1(x);
        }
    }
        temp[index++]=small;

      n--;
    }
    for(int i=index-1;i>=0;i--)
    {
        push1(temp[i]);
    }

  }



void display()
```

```
{
   int i;
   for(i=top1;i>=0;i--)
   {
      printf("%d\t",stack1[i]);
   }
}
int main()
{
   push1(2);

   push1(5);
   push1(3);
   push1(7);
   display();
   printf("\nAfter sorting:");
   sort();
   display();
   return 0;
}
```

```
7       3       5       2
After sorting:2 3       5       7
```

5.the stock span problem is a financial problem where we have a series of daily price quotes for a stock denoted by an array and its task iss to calculate the

spam of the stock price for ith day gives the maxium number of consecutive days leading upto ith day(including current day) where stock price is less than or equal to the price on day.

Eg i/p:[100,80,60,70,60,75,85]

o/p :[1,1,1,2,1,4,6];

```
#include <stdio.h>
#define max 100

void stockspan(int price[] , int n)
{
   int stack[max];
   int span[max];
   int top=-1;

   stack[++top]=0;
   span[0]=1;

   for(int i=1;i<n;i++)
   {
      while(top>=0 && price[stack[top]]<=price[i])
      {
         top--;
      }
```

```
        if(top==-1)
        {
          span[i]=i+1;
        }
        else
        {
           span[i]=i-stack[top];
        }
        span[++top]=i;
    }
    printf("\nspan vaues=");
    for(int i=0;i<n;i++)
    {
        printf("%d\t",span[i]);

    }
}


int main()
{
int price[]={100,80,60,70,60,75,85};
```

```
int n=sizeof(price)/sizeof(price[0]);

stockspan(price,n);


   return 0;

}
```

```
span vaues=1    1       1       2       1       4

...Program finished with exit code 0
Press ENTER to exit console.
```

6. Given an array for each element find the nearest element to its right that has a higher frequency than the current element. If no such element exits set the value no -1 . implement using stack

i/p : [2,1,1,3,2,1]

o/p : [1,-1,-1,2,1,-1]

```
#include<stdio.h>

#define max 100


int top=-1,stack[max];


void ngfe(int arr[] , int n)

{

   //find frequency

   int i,j,count[20];

   int result[20];

   for(i=0;i<n;i++)
```

```
{
   count[i]=0;
}
for(i=0;i<n;i++)
{
      count[arr[i]]++;
  }
  result[n-1]=-1;

  for(i=0;i<n;i++)
  {
     result[i]=-1;
     for(j=i+1;j<n;j++)
     {
        if(count[arr[j]]>count[arr[i]])
        {
           result[i]=arr[j];
           break;
        }
     }
  }
  printf("\nresultant array=");
  for(i=0;i<n;i++)
```

```
    {
      printf("%d\t",result[i]);
    }


}

int main()
{
   int arr[20],n;
   printf("Enter n=");
   scanf("%d",&n);
   printf("\nEnter elements:");
   for(int i=0;i<n;i++)
   {
      scanf("%d",&arr[i]);
   }

   ngfe(arr,n);
   return 0;
}
```

```
Enter n=6

Enter elements:5
1
1
3
5
1

resultant array=1       -1      -1      5       1       -1

...Program finished with exit code 0
```

7.given a pattern containing only I's and d's . I for increasing and D for decreasing . Derive an algorithmic using stack to print minimum members following that pattern. Digits from 1-9 and digits cant repeat

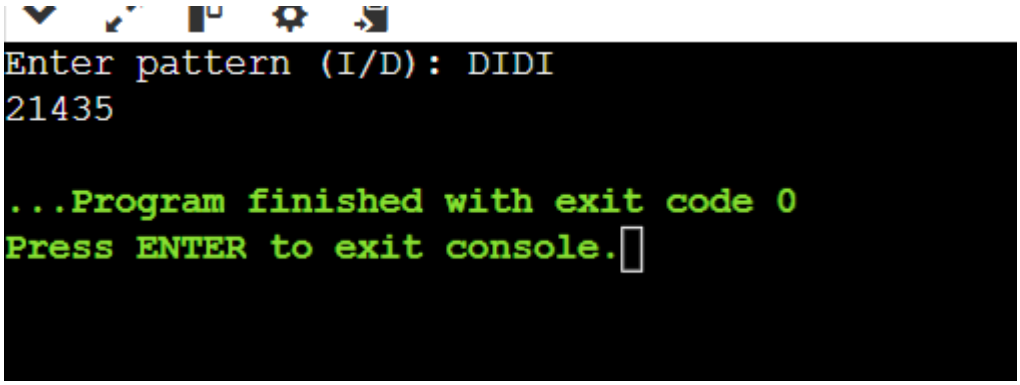
I/p : D o/p 2 1

I/p : DIDI o/p 21435

```
#include <stdio.h>

#define MAX 100


int stack[MAX];

int top = -1;


void push(int x)

{

   stack[++top] = x;

}


int pop()

{
```

```
    return stack[top--];
}

int main()
{
    char pattern[100];
    int i;

    printf("Enter pattern (I/D): ");
    scanf("%s", pattern);

    for(i = 0; pattern[i] != '\0'; i++)
    {
        push(i + 1);

        if(pattern[i] == 'I' )
        {
            while(top != -1)
                printf("%d", pop());
        }
    }

    push(i + 1);
```

```
    while(top != -1)
        printf("%d", pop());

    return 0;
}
```



8.Given an array of digits(values are from 0 to 9) find the minimum possible sum of two members formed from digits of the array . All the digits of given array must be used to from the two members .use queue for implementation

I/P : [6,8,4,5,2,3]

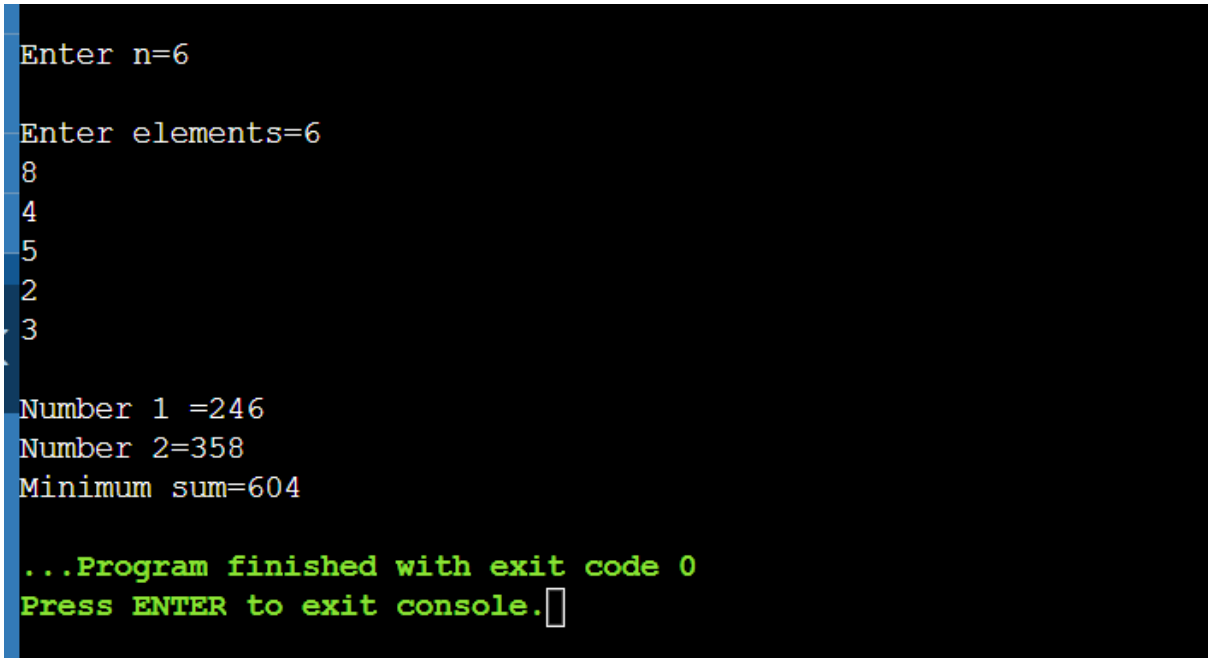
o/p : 604 (min sum formed by no's 358 and 246)

```
#include<stdio.h>

#define max 100


int f1=0,r1=-1;

int f2=0,r2=-1;


int q1[max],q2[max];
```

```
void enqueue1(int x)
{
   q1[++r1]=x;
}

void enqueue2 (int x)
{
   q2[++r2]=x;
}

int main()
{
   int n,a[20];
   int i,j;
   int temp;
   long num1=0,num2=0;
   printf("\nEnter n=");
   scanf("%d",&n);
   printf("\nEnter elements=");
   for(i=0;i<n;i++)
   {
      scanf("%d",&a[i]);
   }
```

```
for(i=0;i<n;i++)

{
   for(j=i+1;j<n;j++)
   {
      if(a[i]>a[j])
      {
         temp=a[i];
         a[i]=a[j];
         a[j]=temp;
      }
   }
}

for(i=0;i<n;i++)
{
   if(i%2==0)
   {
      enqueue1(a[i]);
   }
   else
   {
```

```
            enqueue2(a[i]);
          }
      }

      for(i=f1;i<=r1;i++)
      {
        num1=num1*10+q1[i];
      }
      for(i=f2;i<=r2;i++)
      {
        num2=num2*10+q2[i];
      }

      printf("\nNumber 1 =%d",num1);
      printf("\nNumber 2=%d",num2);
      printf("\nMinimum sum=%d",num1+num2);
      return 0;
  }
```



9.Given an array and positive integer ‘k’ . Find the first negative integer for each window(contiguous subarray) of size k . If a window does not contain an negative integer then print 0 for that window. Implement using queue

Eg: I/P : [-8,2,3,-6,1] k=2

O/p : [-8,0,-6,-6] first negative integer for each window of size 2.

[-8,2] = - 8

[2,3]= 0 (no negative)

[3,-6] = - 6

[-6,1] =-6

#include <stdio.h>

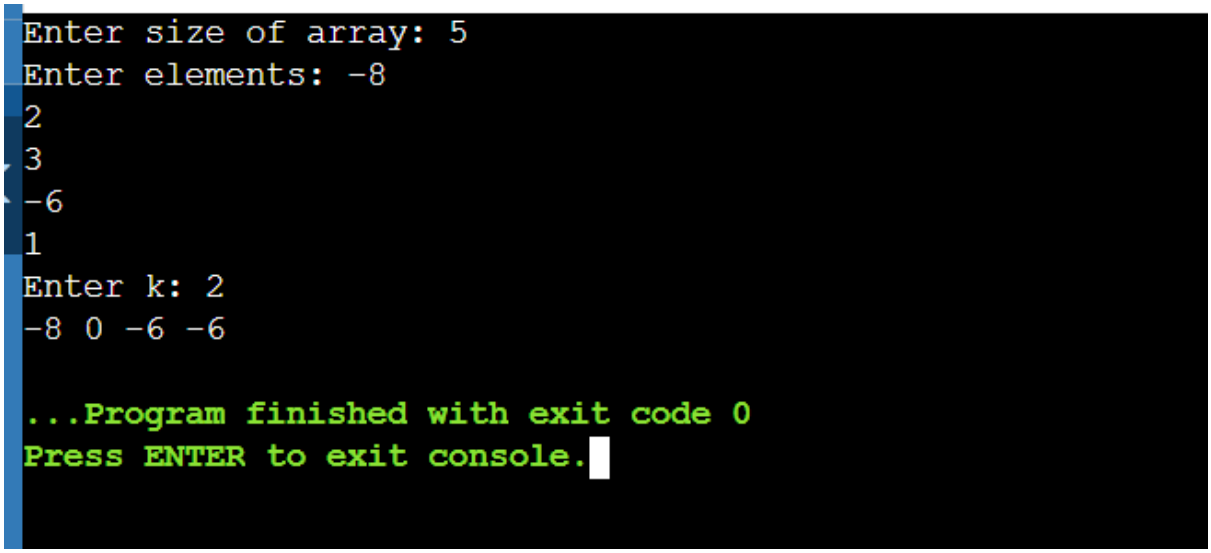
#define MAX 100

int q[MAX];

int front = 0, rear = -1;

```
void enqueue(int x)
{
   q[++rear] = x;
}

void dequeue()
{
   front++;
}

int main()
{
   int arr[MAX], n, k;
   int i;

   printf("Enter size of array: ");
   scanf("%d", &n);

   printf("Enter elements: ");
   for(i = 0; i < n; i++)
      scanf("%d", &arr[i]);

   printf("Enter k: ");
```

```
    scanf("%d", &k);

    for(i = 0; i < n; i++)
    {
        if(arr[i] < 0)
            enqueue(i);

        while(front <= rear && q[front] <= i - k)
            dequeue();

        if(i >= k - 1)
        {
            if(front > rear)
                printf("0 ");
            else
                printf("%d ", arr[q[front]]);
        }
    }

    return 0;
}
```



10) Consider a matrix of dimension M × N where each cell in the matrix can have values 0, 1 or 2, which has the following meaning:

- **0** : Empty cell
- **1** : Cell having fresh oranges
- **2** : Cell having rotten oranges

The task is to find the **minimum time required so that all oranges become rotten**.

A rotten orange at index **(i, j)** can rot fresh oranges which are its **neighbours (up, down, left and right)**.

If it is impossible to rot every orange, then return **-1**.

Use Queue for implementation.

Example

Input:

```
[
 [2,1,0,2,1],
 [1,0,1,2,1],
 [1,0,0,2,1]
]
```

Output:

```
2
```

Explanation

At 0th frame:

```
2 1 0 2 1
```

```
1 0 1 2 1
1 0 0 2 1
```

After 1st frame:

```
2 2 0 2 2
2 0 1 2 2
1 0 0 2 2
```

After 2nd frame:

```
2 2 0 2 2
2 0 2 2 2
2 0 0 2 2
```

All oranges become rotten after **2 units of time**, so the output is:

```
2
```

```
#include <stdio.h>


#define MAX 100


typedef struct
{
   int x;
   int y;
} Node;


Node queue[MAX];
int front = -1, rear = -1;


void enqueue(int x, int y)
{
```

```
    if(front == -1)
        front = 0;

    rear++;
    queue[rear].x = x;
    queue[rear].y = y;
}

Node dequeue()
{
    return queue[front++];
}

int isEmpty()
{
    return front > rear;
}

int main()
{
    int mat[3][5] =
    {
        {2,1,0,2,1},
```

```
    {1,0,1,2,1},
    {1,0,0,2,1}
};

int rows = 3;
int cols = 5;

int fresh = 0;
int time = 0;

// Insert all rotten oranges into queue
for(int i=0;i<rows;i++)
{
    for(int j=0;j<cols;j++)
    {
        if(mat[i][j] == 2)
            enqueue(i,j);

        if(mat[i][j] == 1)
            fresh++;
    }
}
```

```
int dx[] = {-1,1,0,0};
int dy[] = {0,0,-1,1};

while(!isEmpty() && fresh > 0)
{
   int size = rear - front + 1;

   for(int k=0;k<size;k++)
   {
      Node curr = dequeue();

      for(int d=0;d<4;d++)
      {
         int nx = curr.x + dx[d];
         int ny = curr.y + dy[d];

         if(nx>=0 && nx<rows &&
           ny>=0 && ny<cols &&
           mat[nx][ny]==1)
         {
            mat[nx][ny]=2;
            fresh--;
```

```
                enqueue(nx,ny);
            }
        }
    }

    time++;
  }

  if(fresh == 0)
    printf("Minimum Time = %d\n", time);
  else
    printf("-1\n");

  return 0;
}
```

```
Minimum Time = 2


...Program finished with exit code 0
Press ENTER to exit console.
```